Efficient Federated LLM Framework for Intelligent IoMT Management

Yazan Otoum, Syed Muhammad Danish, Ishtiaq Ahmad, Yazeed Alkhrijah, and Arghavan Asad

***Abstract*—The rapid expansion of Internet of Medical Things (IoMT) deployments has introduced significant challenges in scalability, security, and real-time data analytics. Conventional centralized learning architectures are increasingly impractical due to high latency, privacy concerns, and elevated communication overhead. To address these challenges, this paper proposes a Federated Learning-enabled Large Language Model (LLM) framework that supports intelligent, privacy-preserving analytics in IoMT environments. The framework targets emerging Graphics Processing Unit (GPU)-equipped edge platforms, such as smart medical gateways and hospital fog nodes, to enable on-site LLM inference and lightweight adaptation via prompt tuning and partial fine-tuning, thereby supporting edge-scale healthcare applications. The proposed framework integrates transformer-based LLMs within a hybrid edge–cloud architecture that jointly balances on-device intelligence, communication efficiency, and centralized coordination for scalable IoMT analytics. It introduces an Enhanced Gradient Sensing Federated Strategy (Enhanced GSFS) that dynamically optimizes model updates in response to real-time network conditions. Enhanced GSFS incorporates two key optimization mechanisms: (i) adaptive thresholding based on real-time performance metrics, and (ii) priority-aware client scheduling guided by historical contribution tracking. These enhancements collectively improve convergence behaviour and communication efficiency under federated constraints. Extensive experiments across two benchmark IoT datasets, IoT-23 and ToN IoT, demonstrate that the proposed FL-LLM framework achieves superior performance, reducing average inference latency by 5.47% and improving energy efficiency by 9.25% compared to state-of-the-art FL methods, including Federated Averaging (FedAvg), Federated Optimization (FedOpt), and Gradient Sensing Federated Strategy (GSFS).**

***Index Terms*—Large Language Models (LLMs), Internet of Medical Things (IoMT), Federated Learning (FL), Edge-cloud Aggregation, Gradient-sensing Federated Strategy.**

I. Introduction

THE rapid proliferation of Internet of Medical Things (IoMT) ecosystems has led to a surge in connected devices, generating vast amounts of heterogeneous data. Managing and optimizing these distributed systems poses substantial challenges, particularly in real-time data processing, resource allocation, security, and energy efficiency. Traditional cloud-centric approaches struggle to meet the demands of modern IoMT workloads due to scalability bottlenecks, latency issues, and critical privacy concerns, particularly in sensitive domains such as clinical care and hospital operations, medical imaging pipelines, and device-integrated healthcare workflows [1], [2]. These limitations underscore the need for decentralized, intelligent, and adaptive frameworks that can operate effectively in dynamic, distributed IoMT environments. Recent advances in artificial intelligence, especially the emergence of Large Language Models (LLMs), have demonstrated transformative capabilities in natural language understanding, predictive analytics, and decision-making. In IoMT contexts, LLMs can interpret complex telemetry, detect anomalies, and enable context-aware automation [3].

However, integrating LLMs into distributed IoMT environments poses several challenges, including high computational demands, real-time responsiveness, and stringent data privacy constraints. Without appropriate adaptation, the direct deployment of LLMs can lead to communication inefficiencies and increased energy consumption [4]. Federated Learning (FL) has emerged as a promising paradigm for addressing these challenges by enabling decentralized model training across edge devices without exchanging raw data. By aggregating local model updates centrally, FL enhances privacy and scalability while reducing communication overhead [5], [6]. As IoMT deployments increasingly incorporate Graphics Processing Unit (GPU)-enabled edge infrastructure, such as smart medical hubs, hospital gateways, and clinical fog computing nodes, new opportunities arise to execute LLMs directly at the edge [7]. These resource-rich platforms support advanced local inference and fine-tuning, enabling semantic reasoning, anomaly detection, and real-time decision-making close to the data source while preserving the locality of Protected Health Information (PHI). This shift toward capable edge environments motivates the development of hybrid learning architectures that can harness the strengths of both LLMs and FL while preserving data locality and optimizing latency and energy performance. Yet, despite rapid progress, practitioners lack a systematic comparison of LLM families for IoMT security and clinical telemetry under federated constraints; a clear benchmark is needed to guide model selection and deployment.

To this end, we propose a federated LLM deployment framework built upon a hybrid edge–cloud architecture. LLMs are distributed across edge and cloud tiers to enable intelligent decision-making for IoMT security classification and anomaly detection, while FL provides scalable, privacy-

Y. Otoum, S. M. Danish, and A. Asad are with the School of Computer Science and Technology, Algoma University, ON, Canada (e-mail: {otoum, syed.danish, arghavan.asad}@algomau.ca).

I. Ahmad is with the Faculty of Electrical Engineering, Czech Technical University in Prague, Prague, Czech Republic (e-mail: ahmadish@fel.cvut.cz).

Y. Alkhrijah is with the Department of Electrical Engineering, College of Engineering, Imam Mohammad Ibn Saud Islamic University (IMSIU), Riyadh 11564, Saudi Arabia, (e-mail: Ymalkhrijah@imamu.edu.sa; maawaad@imamu.edu.sa)

This work was supported and funded by the Deanship of Scientific Research at Imam Mohammad Ibn Saud Islamic University (IMSIU) (grant number IMSIU-DDRSP26041).

preserving learning across participating devices. At the core of our framework is a novel performance-aware coordination mechanism, the Enhanced Gradient Sensing Federated Strategy (Enhanced GSFS), which dynamically regulates client participation and model updates based on real-time performance metrics, namely, accuracy, precision, recall, F1-score, inference latency, and energy consumption. Our baselines comprise Federated Averaging (FedAvg), which performs simple unweighted averaging of client updates; Federated Optimization (FedOpt), which applies server-side adaptive optimization of the global model using methods such as Adam or Yogi; and Gradient Sensing Federated Strategy (GSFS), which schedules clients based on gradient sensing without performance-aware priority scoring. This strategy minimizes communication overhead while ensuring rapid convergence and high predictive accuracy in heterogeneous, distributed IoMT environments. The paper contributes the following:

- We present the first comprehensive empirical benchmark of 17 transformer-based LLMs across six model families, evaluating their suitability for federated IoMT security classification and clinical device telemetry monitoring.
- We propose Enhanced Gradient Sensing Federated Strategy (Enhanced GSFS), a novel performance-aware federated learning mechanism that dynamically controls client participation using accuracy-based triggers, gradient-norm sensing, and priority-aware scheduling, reducing communication overhead while improving latency and energy efficiency compared to FedAvg, FedOpt, and GSFS.
- We develop a scalable, privacy-preserving FL–LLM architecture for IoMT security that enables edge-level prompt adaptation and cloud coordination while protecting PHI and model-update integrity. Experiments on IoT-23 and ToN_IoT demonstrate their effectiveness under non-IID conditions and reveal key trade-offs in accuracy, latency, and energy efficiency.

The remainder of this paper is structured as follows: Section II reviews related work on federated learning, LLMs, and their applications in IoMT environments. Section III introduces the proposed LLM-FL hybrid framework, detailing its key components and adaptive learning strategies. Section IV presents the performance analysis, comparing our approach against baseline models. Finally, Section V concludes the paper by summarizing our findings and outlining future research directions.

II. State-of-the-Art Literature Review

Recent advancements in FL, LLMs, and IoMT security have accelerated the development of intelligent, privacy-preserving distributed systems. However, the rapid evolution of these research directions has led to fragmented progress, with each line of work addressing isolated components of the broader challenge of deploying transformer-based models in heterogeneous, resource-constrained IoT environments. To situate our contributions within this landscape, this section synthesizes the most relevant literature and highlights the gaps that motivate the design of our unified FL-LLM architecture. A substantial body of research has explored FL for IoMT security and analytics. The authors of [11] develop a standardization and benchmarking framework for FL-based intrusion detection systems in IoMT, using multicriteria decision-making to rank candidate ML-based IDS models on both security and performance, with CPU time identified as the dominant efficiency criterion. The authors of [12] design a federated blockchain–IoT framework for sustainable Healthcare 5.0 that integrates FL with blockchain-backed IoMT sensing to enable secure health monitoring and intrusion detection. In [13], the authors propose FED-EHR, a privacy-preserving FL framework for decentralized EHR analytics that emphasizes secure aggregation and robustness to institutional data silos. Complementary IoMT-focused efforts target continuous sensing and activity monitoring: the work in [14] introduces a federated human-activity-recognition pipeline built on hybrid LSTM–GRU models for wearable IoMT devices; the authors of [15] present a fused FL architecture combining RTS-DELM and homomorphic encryption for chronic kidney disease prediction over heterogeneous IoMT streams; and the work in [16] employs FL on smartphone-based edge devices to support real-time elderly activity and context monitoring. These studies demonstrate the potential of FL to enhance privacy-preserving intelligence in IoMT environments. Yet, they predominantly rely on conventional deep models rather than on transformer-based LLMs, and they do not address prompt-level adaptation or performance-aware coordination across GPU-enabled edge clients. The FLoRA framework [17] introduces heterogeneous low-rank adaptations to support efficient LLM fine-tuning with reduced bandwidth requirements, while OpenFedLLM [18] demonstrates decentralized LLM training with strong privacy guarantees. Continual learning has also been considered, as in [19], which applies federated continual adaptation to industrial IoT tasks. Comprehensive surveys such as [20] further classify emerging FL–LLM techniques and identify critical challenges, including personalization, data heterogeneity, and edge resource constraints. Although these efforts push LLMs toward decentralized intelligence, most lack empirical evaluation of latency, energy efficiency, and the feasibility of real-world edge deployment, critical factors for IoT and IoMT environments.

Parallel efforts have examined semantic intelligence, trust, and zero-trust architectures in IoT security, where prompt engineering and LLM-based reasoning are increasingly influential. For instance, [21] introduces a cross-domain zero-trust FL architecture that strengthens authentication and authorization but does not incorporate semantic reasoning via LLMs. The ZTFM framework [22] blends blockchain-backed identity verification, trusted execution environments, and pre-trained transformers to support trust evaluation, yet omits evaluation on edge-limited hardware. Other works integrate blockchain or physical-constraint checks to improve decision reliability [23], [24]. While these contributions enhance trustworthiness and adaptivity, they do not address real-time prompt optimization or resource-sensitive inference, which are essential for LLMs to operate at the edge. Complementary research focuses on generative IoT intelligence and emerging LLM-enhanced architectures. Surveys such as [25] highlight the promise of

TABLE I: Comparative Analysis of Recent FL+LLM Frameworks

Work	FL+LLM Context	Client Device Type	Optimization Strategy	Comm. Efficiency	GPU Edge Support
FATE-LLM [8]	Federated tuning of GPT/LLaMA via PEFT	Server-grade clients	Parameter-efficient tuning (LoRA, adapters)	High	✗
FwdLLM [9]	Transformer-based FL tuning (BERT, RoBERTa)	General NLP clients	Fine-tuning + pruning	High	✗
KOALA [10]	LLM distillation for FL on IoT	Resource-constrained IoT clients	Teacher–student distillation	Moderate	✗
Ours	FL with LLM prompt adaptation (17 models)	GPU-based edge (e.g., fog, gateways)	Enhanced GSFS + prompt adaptation	High	✔

generative AI for critical infrastructure while emphasizing unresolved challenges, including energy constraints and adversarial robustness. More practical implementations, such as the BERT-based FL intrusion detection system in [26], apply quantization and efficient training for 5G-enabled IoT but evaluate only mid-sized transformers and omit latency or energy measurements. Recent efforts, such as [27], aim to advance FL–LLM convergence by incorporating gradient sensing and Generative IoT (GIoT) orchestration; however, they fall short of providing unified evaluations of performance, efficiency, and deployment complexity. These works reveal a clear need for a cohesive FL–LLM framework that operates on GPU-enabled edge platforms, supports dynamic prompt-based adaptation, and delivers verifiable efficiency gains under realistic IoT and IoMT constraints. To address these gaps, our work contributes: (i) a unified architecture that integrates transformer-based LLMs with federated optimization and real-time prompt adaptation; (ii) an Enhanced Gradient Sensing Federated Strategy (Enhanced GSFS) that provides adaptive thresholding and priority-aware client coordination; and (iii) an extensive empirical benchmark of 17 LLMs across two representative IoT datasets under practical GPU-enabled edge settings. This unified perspective enhances the feasibility of deploying secure, scalable, and efficient LLMs across smart gateways, industrial fog nodes, and emerging medical AI platforms.

III. Proposed Framework

A. Architecture Overview

The proposed framework combines edge computing, federated learning, and cloud-based orchestration to enable a scalable, adaptive IoMT management architecture. It comprises the following components: the Local LLM Training Module, Federated Learning Coordinator, Communication Module, Prompt Management Module, Adaptive Feedback Loop, and Performance and Monitoring Dashboard. The details are as follows:

1) Local LLM Training Module: This component runs on edge devices, such as hospital gateways, which serve as near-patient computational nodes that locally process medical telemetry to minimize latency and preserve data locality [28]. In parallel, clinical fog nodes serve as an intermediate computing tier between the edge and the cloud, providing higher computational capacity for real-time analytics, data filtering, secure aggregation, and other latency-sensitive healthcare tasks [29]. This component runs on edge devices (e.g., hospital gateways and clinical fog nodes with moderate computational capabilities, including GPU acceleration) and processes medical device telemetry, Electronic Health Record (EHR)- linked system logs, and real-time clinical sensor data. It supports federated training, enabling distributed model updates while preserving data privacy by keeping raw data local in line with Health Insurance Portability and Accountability Act and Personal Health Information Protection Act HIPAA/PHIPA requirements. To enhance efficiency and responsiveness, the component leverages lightweight prompt tuning/LoRA-based fine-tuning, as well as knowledge distillation, a technique in which a smaller "student" model is trained to replicate the behaviour of a larger, more complex "teacher" model [30]. In our implementation, lightweight adaptation is achieved using LoRA with adapter rank $r = 8$, applied to the transformer attention projection matrices (Query, Key, Value, and Output) and to the feed-forward (MLP) layers. This configuration updates only a small fraction of the model parameters (typically less than 1–2% of those in full fine-tuning), enabling efficient on-device adaptation while preserving model performance in resource-constrained IoMT edge environments. This design reduces training time and model footprint without significantly compromising performance, enabling scalable, privacy-preserving inference at the hospital network edge.

2) Federated Learning Coordinator: Hosted in the cloud, this module aggregates model updates from distributed edge nodes while ensuring data privacy and security. It leverages secure aggregation mechanisms, including Differential Privacy (DP-FedAvg) [31] and Gradient Compression, to reduce communication overhead. Periodic synchronization facilitates system-wide adaptability and mitigates the effects of non-IID data distributions.

3) Communication Module: This module manages secure and efficient data exchange between edge and cloud components. It incorporates adaptive data-compression techniques to reduce communication overhead. It employs explicit bandwidth-allocation strategies, including dynamic rate control and priority-based scheduling, to maintain reliable performance under varying network conditions. In addition, it uses Transport Layer Security (TLS) encryption to ensure data integrity and confidentiality during transmission [32].

4) Prompt Management Module: This module dynamically adapts prompts according to the capabilities of the executing node, including resource-constrained IoMT devices, hospital gateways, and GPU-enabled clinical fog servers. It further accounts for the operational context, including current network latency, available bandwidth, and node-level workload, as well as application-specific requirements, such as accuracy

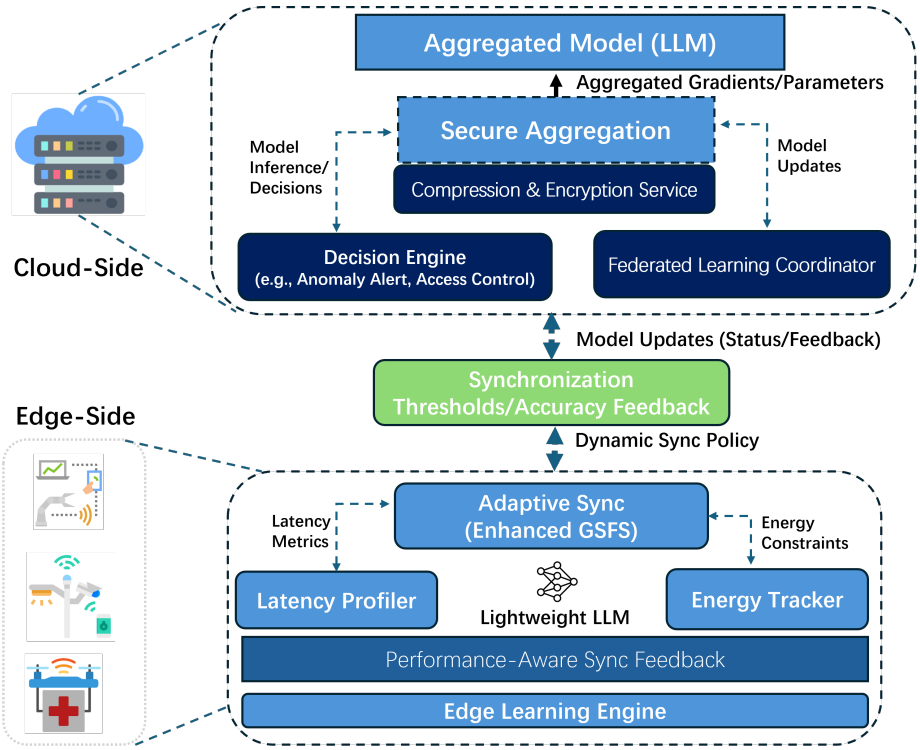


Fig. 1: Overview of the proposed model. The system integrates edge-level LLM training, cloud-based model aggregation, and secure communication to enable scalable, low-latency, and privacy-preserving intelligence.

targets and response-time constraints. The module interacts with the Adaptive Feedback Loop to incorporate runtime performance signals, including prompt effectiveness, token-usage efficiency, and model response quality. Reinforcement learning is employed to continuously refine prompt selection and compression strategies based on observed rewards that jointly capture task performance, latency, and token budget, thereby enabling adaptation to changing network conditions and clinical user requirements [33], [34]. To reduce inference cost while maintaining accuracy, the module incorporates prompt compression and token-efficiency techniques [35]. In addition, aggregated insights from the Performance and Monitoring Dashboard, including end-to-end inference latency, token consumption, throughput, and resource utilization, are used to further optimize prompt templates and runtime routing decisions. Prompts are locally optimized at each edge client to reflect node-level context and workload conditions, and are not globally aggregated. Federated learning is applied to model parameters (or lightweight LoRA adapters), while the RL controller adjusts both synchronization thresholds and prompt compression/selection policies based on runtime state variables (accuracy, latency, energy, bandwidth, and update frequency). This design enables personalized prompt adaptation at the edge while maintaining global model consistency through federated aggregation.

5) Adaptive Feedback Loop: This component continuously monitors key indicators, including model accuracy, latency, energy consumption, and prompt-response quality. Reinforcement learning (RL) is used to adapt both prompt configurations and federated learning parameters. Formally, the adaptive feedback loop is modelled as an RL agent interacting with the federated IoMT environment. At each federated round t, the agent observes the system state:

$$s_t = \{Acc_t, F1_t, L_t, E_t, B_t, U_t\}, \tag{1}$$

where Acc_t and $F1_t$ denote the global accuracy and F1-score, L_t is inference latency, E_t is energy consumption, B_t is available bandwidth, and U_t represents the client update frequency. Based on s_t, the agent selects an action:

$$a_t = \{\delta_t, K_t, \eta_t, c_t\}, \tag{2}$$

where δ_t is the synchronization threshold, K_t is the number of selected clients, η_t is the learning rate adjustment factor, and c_t denotes the prompt compression level. The reward function is defined to maximize prediction performance while minimizing latency and energy cost:

$$r_t = \alpha Acc_t + \beta F1_t - \gamma L_t - \mu E_t, \tag{3}$$

where α, β, γ, and μ are weighting coefficients. The objective is to learn an optimal policy π^* that maximizes the expected discounted cumulative reward:

$$\pi^* = \arg\max_{\pi} E\left[\sum_{t=1}^{T} \lambda^t r_t\right], \tag{4}$$

where $\lambda \in (0, 1)$ is the discount factor. This RL-based feedback mechanism enables dynamic adaptation of synchronization and prompt strategies under heterogeneous IoMT operating conditions.

Client selection is governed by a performance-aware policy that considers factors such as recent model improvements, gradient-stability metrics, communication frequency, and the

current resource availability of each participating node. The loop functions as a control layer that generates optimization signals, covering learning-rate adjustments, aggregation-frequency updates, prompt-compression levels, and participation priorities. It transmits them to both the Prompt Management Module and the Federated Learning Coordinator. Coordination occurs through an internal messaging interface that synchronizes these components, ensuring prompt adaptation and real-time, jointly tuned scheduling of federated updates, thereby enabling a closed-loop mechanism for continuous, system-wide self-optimization across the architecture. This component continuously monitors key system metrics, including model accuracy, inference latency, energy consumption, and prompt-response quality. Based on these metrics, a reinforcement-learning–based controller adjusts both prompt configurations and federated learning parameters. Client selection is performed using a performance-aware policy that prioritizes nodes with stable gradients, recent accuracy improvements, sufficient resources, and acceptable communication frequency. The feedback loop generates optimization signals, including updates to the learning rate, adjustments to the aggregation frequency, prompt compression levels, and client participation priorities. These signals are communicated through an internal coordination interface to both the Prompt Management Module and the Federated Learning Coordinator, ensuring synchronized adaptation of local inference behaviour and global model aggregation. This closed-loop design enables continuous, system-wide self-tuning across the federated IoMT architecture.

6) Performance and Monitoring Dashboard: The dashboard provides a unified interface for system administrators, including hospital IT operators, clinical data engineers, and security managers, to observe real-time system metrics, such as model drift, training-convergence behaviour, latency patterns, and detailed prompt-performance statistics, including response quality, failure frequency, and token consumption. These analytics are fed into the Adaptive Feedback Loop, enabling data-driven adjustments that refine prompt templates, update parameter schedules, and improve the relevance and efficiency of model behaviour across diverse IoMT workloads. The dashboard provides a unified interface for system administrators to observe real-time system metrics, including model drift, training convergence, latency patterns, and prompt performance statistics (e.g., response quality, failure rates, and token consumption). These analytics support data-driven adjustments through the Feedback Loop and help refine prompt templates over time, improving task relevance and efficiency.

B. IoTM Datasets

Although these datasets are not collected directly from hospital IoMT infrastructures, they serve as reliable proxies because modern IoMT deployments share fundamental communication and threat characteristics with general IoT systems, including distributed device telemetry, gateway-mediated traffic, heterogeneous log sources, and malware-driven intrusion behaviours. In particular, IoT-23 captures real malware-based IoT traffic flows that resemble medical device monitoring and remote-control scenarios, while ToN-IoT provides multi-source telemetry (network traffic, system logs, and device activity traces) that closely reflects the operational structure of IoMT ecosystems. Therefore, these datasets enable reproducible evaluation of intrusion detection and anomaly classification under realistic, non-IID, distributed conditions, which are central challenges in federated IoMT environments.

To assess the robustness and generalizability of our proposed FL–LLM framework, we evaluate it on two widely used IoT security datasets: IoT-23 [36] and ToN_IoT [37]. These datasets provide complementary perspectives, including network-level intrusion traces and multi-source telemetry, enabling benchmarking across diverse threat scenarios. The IoT-23 dataset contains labelled network traffic flows derived from realistic malware activity on IoT devices. After removing missing values and resolving encoding inconsistencies through label encoding and normalization, we obtained a cleaned dataset containing 48,003 samples, which were split into 70% training, 20% validation, and 10% testing subsets (Table II). The ToN_IoT dataset, developed by UNSW Sydney, integrates network traffic, system logs, and telemetry collected from heterogeneous IoMT and IIoT environments. We selected the structured `Train_Test_IoT` subset, commonly used for anomaly and malware detection, which yielded 50,303 cleaned samples, partitioned using the same ratio (Table III). To prepare both datasets for ingestion by transformer-based LLMs, we adopted a two-stage preprocessing pipeline. First, categorical features were encoded using one-hot or label encoding, and continuous variables were normalized via min-max scaling. Then, structured records were serialized into tokenized sequences using fixed formatting strategies and domain-specific embedding templates. This transformation enabled compatibility with text-based LLM architectures while preserving the semantic and structural properties of the original IoT telemetry and traffic data. These adaptations ensured uniformity across the FL-LLM training pipeline and supported consistent evaluation of predictive performance, inference latency, and energy consumption under federated constraints.

TABLE II: IoT-23 Dataset Split into Training, Validation, and Testing Sets

Dataset Part	Percentage (%)	Number of Samples
Training Set	70	33,602
Validation Set	20	9,601
Testing Set	10	4,800

TABLE III: ToN_IoT Dataset Split into Training, Validation, and Testing Sets

Dataset Part	Percentage (%)	Number of Samples
Training Set	70	35,213
Validation Set	20	10,060
Testing Set	10	5,030

C. Implementation Details

The implementation follows a hierarchical execution flow that begins at the edge, moves upward through fog coordination, and ultimately integrates into cloud-level aggregation.

At the first stage, each edge device runs a Local Inference and Adaptation Engine built on a size-optimized LLM. This engine performs real-time functions, such as anomaly detection, dynamic resource allocation, and predictive maintenance, directly on the device, thereby reducing the need to transmit raw telemetry to the cloud. Executing these operations locally minimizes latency, reduces bandwidth usage, and improves energy efficiency, ensuring that time-sensitive IoMT decisions are made as close as possible to the data source.

1) Edge Processing: Once edge-level inference and lightweight adaptation are complete, the system transitions to the federated learning phase, in which updates from multiple edge devices must be synchronized and aggregated. This motivates the need for robust federated learning baselines to manage distributed optimization and ensure model consistency across heterogeneous IoMT nodes. The following subsection introduces the classical FL strategies used as benchmarks in our framework. Each edge device is equipped with a Local Inference and Adaptation Engine that leverages trained LLMs for real-time decision-making. This enables critical tasks such as dynamic resource allocation, predictive maintenance, and anomaly detection without relying on cloud resources. By performing computations locally, the framework minimizes latency, reduces bandwidth consumption, and enhances overall energy efficiency, making it well-suited for IoT environments.

2) Classical Federated Learning Baselines: Classical federated learning methods, such as FedAvg, serve as a reference for evaluating the effectiveness of our proposed strategy. These baseline algorithms represent the standard behaviour of federated optimization, uniform client participation, synchronous aggregation, and unweighted averaging, against which improvements in convergence, communication efficiency, and accuracy can be measured.

a) Federated Averaging (FedAvg): is one of the most classical and widely used federated learning strategies. Each client k maintains a local model $\theta_{k,t}$ and updates it by running E local epochs of Stochastic Gradient Descent (SGD) on its private dataset D_k with learning rate η. Formally, after local training, each client has

$$\theta_{k,t+1} \;=\; \theta_{k,t} - \eta\,\nabla\mathcal{L}(\theta_{k,t}, D_k). \tag{5}$$

repeated for E epochs. All participating edge or fog clients then send their updated parameters (or gradients) to the central server, which performs a global aggregation. If there are N clients in total and client k's dataset size is $|D_k|$, a common aggregation is the weighted average:

$$\theta_{\text{global}}^{(t+1)} \;=\; \frac{1}{\sum_{k=1}^{N}|D_k|}\sum_{k=1}^{N}|D_k|\;\theta_{k,t+1}. \tag{6}$$

Finally, the new global model is broadcast back to each client, completing one federated round.

b) FedOpt.: FedAvg may converge slowly or become unstable when clients hold heterogeneous (non-IID) data. FedOpt addresses this limitation by applying adaptive optimization on the server, thereby improving convergence speed, enhancing stability, and yielding more reliable global-model updates in federated environments. FedOpt is a family of methods that extend FedAvg by using an *adaptive optimizer* on the server side. Instead of simple averaging, the server updates the global model $\theta_{\text{global}}^{(t)}$ using certain rules, here, for instance, an Adam-like rule:

$$g_t \;=\; \frac{1}{N}\sum_{k=1}^{N}\Delta\theta_{k,t}. \tag{7}$$

$$m_{t+1} = \beta_1 m_t + (1-\beta_1) g_t. \tag{8}$$

$$v_{t+1} = \beta_2 v_t + (1-\beta_2) g_t^2. \tag{9}$$

$$\theta_{\text{global}}^{(t+1)} \;=\; \theta_{\text{global}}^{(t)} - \eta_{\text{server}}\,\frac{m_{t+1}}{\sqrt{v_{t+1}}+\epsilon}. \tag{10}$$

where m_t and v_t are first and second moment estimates, similar to those used in Adam and β_1 and β_2 are exponential decay coefficients that control how much past gradients influence the first- and second-moment estimates, g_t denotes the aggregated gradient update received from participating clients at round t, and ϵ is a small constant added for numerical stability to avoid division by zero.

3) Gradient Sensing Federal Strategy: Classical federated learning methods such as FedAvg require every client to upload its full model update at the end of each training round, regardless of the update's magnitude. This uniform and frequent synchronization results in high communication overhead, increased latency, and inefficient use of bandwidth. These issues become more severe as the number of clients grows or devices operate over constrained IoMT networks. GSFS addresses this limitation by allowing clients to asynchronously upload updates only when meaningful local changes occur. Instead of sending updates every round, each client monitors indicators such as gradient magnitude or performance variation and initiates an upload only when they exceed predefined thresholds. This selective-update mechanism significantly reduces unnecessary communication, lowers latency, and maintains convergence quality, making GSFS better suited for large-scale, bandwidth-limited IoMT environments.

Most classical federated learning algorithms (e.g., FedAvg) require each client to upload its complete update (either model parameters or gradients) to the central server after each local training round. The server then performs global aggregation (e.g., simple or weighted averaging) and broadcasts the updated global model to all clients. This frequent and uniform synchronization results in high communication overhead and increased latency, particularly when the number of participating devices is large or network bandwidth is constrained. The GSFS minimizes unnecessary communication by enabling clients to *asynchronously trigger* uploads in response to significant locally observed changes. The core idea is to monitor the magnitude of updates or performance shifts in local models and initiate uploads only when a predefined *threshold* is exceeded. The following section outlines the details of this approach from both the client and server perspectives.

a) Client Side:

To avoid unnecessary communication, each client must determine whether its local model has changed enough to justify

TABLE IV: Federated Learning Hyperparameters Used

Parameter	Value
Number of clients (N)	5
Federated rounds (T)	50
Local epochs (E)	5
Batch size (B)	32
Client optimizer	Adam
Client learning rate (η)	10^{-4}
Server optimizer (FedOpt)	Adam
Server learning rate (η_{server})	10^{-3}
Aggregation strategy	FedAvg / FedOpt / GSFS / Enhanced GSFS
Aggregation threshold (GSFS-based)	60% of client updates

sending an update to the server. A performance-based trigger provides this decision mechanism by allowing the client to monitor its own progress and upload updates only when its local performance meaningfully improves or deteriorates. This ensures that communication occurs only when it is helpful for global training, reducing bandwidth usage and synchronization overhead. Each client k maintains a local model $\theta_{k,t}$ and a local validation dataset D_k^{val}. After several local mini-batch updates or at periodic intervals, the client evaluates its model on D_k^{val} to obtain a performance metric (e.g., accuracy, loss, or F1 Score). Let $\mathcal{M}(\theta_{k,t}; D_k^{\text{val}})$ denote the chosen metric for client k at time t. The client compares the current performance $\mathcal{M}(\theta_{k,t}; D_k^{\text{val}})$ with the performance at the last upload reference point $\mathcal{M}(\theta_{k,t_{\text{ref}}}; D_k^{\text{val}})$:

$$\Delta\mathcal{M}_k \;=\; \Big|\, \mathcal{M}(\theta_{k,t}; D_k^{\text{val}}) \;-\; \mathcal{M}(\theta_{k,t_{\text{ref}}}; D_k^{\text{val}}) \Big|. \tag{11}$$

If $\Delta\mathcal{M}_k$ exceeds an adaptive threshold δ_k^{perf}, the client initiates an *asynchronous upload* of its gradients updates to the server:

$$\text{Trigger:} \quad \Delta\mathcal{M}_k \;>\; \delta_k^{\text{perf}}. \tag{12}$$

In addition to using performance metrics, clients can monitor the norm of local gradients or parameter changes across training steps. Denote by $g_k^{(l)}$ the aggregated gradient norm at layer l of client k. A moving average (mean) $\mu_k^{(l)}$ and standard deviation $\sigma_k^{(l)}$ can be maintained over recent training steps:

$$\mu_k^{(l)} \leftarrow \alpha\, \mu_k^{(l)} \;+\; (1-\alpha)\, \|\, g_k^{(l)} \|, \tag{13}$$

$$\sigma_k^{(l)} \leftarrow \alpha\, \sigma_k^{(l)} \;+\; (1-\alpha) \Big|\, \|\, g_k^{(l)} \| \;-\; \mu_k^{(l)} \Big|, \tag{14}$$

where $\alpha \in (0,1)$ is a smoothing factor. An adaptive threshold $\delta_k^{(l)}$ can then be defined for each layer:

$$\delta_k^{(l)} \;=\; \mu_k^{(l)} \;+\; \beta\, \sigma_k^{(l)}, \tag{15}$$

where β is a hyperparameter that controls sensitivity. If any layer's gradient norm $\|\, g_k^{(l)} \|$ exceeds $\delta_k^{(l)}$, client k triggers an upload.

b) Server Side

When the central server receives a gradient update or parameter update $\Delta\theta_{k,t}$ from client k, it stores it in an *update pool* along with metadata such as timestamp or training round:

$$\text{Pool} \;=\; \Big\{\, (\Delta\theta_{k,t},\, \text{time}_{k,t},\, k) \Big\}. \tag{16}$$

Once the number of distinct clients, |Pool|, reaches the threshold M (defaulting to 60% of all clients), the server performs a global aggregation step. Denoting by $\mathcal{K}_t$ the set of clients whose updates are included at aggregation time t, the form of the aggregation can be:

$$\theta_{\text{global}}^{(t+1)} \;=\; \text{Aggregate}\Big(\theta_{\text{global}}^{(t)},\, \{\Delta\theta_{k,t} : k \in \mathcal{K}_t\}\Big). \tag{17}$$

We adopt a common choice to the weighted sum of the updates while using the submission frequency as the weights for calculation, for instance:

$$\theta_{\text{global}}^{(t+1)} \;=\; \theta_{\text{global}}^{(t)} \;+\; \sum_{k \in \mathcal{K}_t} \alpha_k\, \Delta\theta_{k,t}, \tag{18}$$

where α_k reflects the importance of client k (e.g., positively proportional to k's submission frequency). After the server aggregates the updates in the pool to form a new global model $\theta_{\text{global}}^{(t+1)}$, it broadcasts this model to *all* clients simultaneously:

$$\text{Broadcast:} \quad \forall k, \quad \theta_k^{(t+1)} \leftarrow \theta_{\text{global}}^{(t+1)}. \tag{19}$$

Each client receives the latest version of the global model, ensuring *download consistency*. Clients will then reset their local reference models (e.g., $\theta_{k,t_{\text{ref}}} \leftarrow \theta_k^{(t+1)}$) for subsequent local training and trigger detection. Overall, the GSFS allows each client to upload *asynchronously* upon detecting significant changes (see 12). Still, the server consolidates those asynchronous updates into a *single broadcast* event. Regardless of how frequently a client uploads, it always receives the latest global model at each broadcast iteration. This design reduces communication overhead compared to forcing uniform uploads from all clients after every local epoch.

c) Cloud Aggregation: The Federated Learning Coordinator aggregates updates from edge devices using privacy-preserving algorithms such as Federated Averaging. The global model leverages cloud computing for large-scale updates, ensuring scalability and robustness. Communication between the central server (or aggregator) and edge clients is a critical component of federated learning frameworks. It must handle model updates, client feedback, and synchronization signals robustly and securely. In our system, we use a TCP/IP-based Transport Protocol, widely adopted for its simplicity and cross-platform compatibility. The central server and clients exchange model parameters over standard TCP sockets secured with TLS/SSL to ensure data confidentiality and integrity.

4) Optimization Strategies for Adaptive Synchronization: To further optimize our GSFS, we propose two innovative enhancements focusing on communication efficiency and adaptive synchronization: *a) Differential Threshold Adjustment:* Instead of using fixed thresholds for gradient norm triggers, we introduce an adaptive threshold mechanism based on the historical performance trends of local devices. Specifically, each client dynamically adjusts its threshold $\delta_k^{(l)}$ using an exponentially weighted moving average of recent improvements in accuracy and communication frequency. Formally, this threshold adjustment is defined as:

$$\delta_{k,t}^{(l)} = \gamma \cdot \delta_{k,t-1}^{(l)} + (1-\gamma) \cdot (\lambda \cdot \Delta\mathcal{M}k + \mu \cdot \rho_k), \tag{20}$$

where $\delta k, t^{(l)}$ is the threshold at round t for layer l and client k, γ is the smoothing parameter, $\Delta\mathcal{M}_k$ is the recent accuracy

improvement, ρ_k is the communication frequency, and λ, μ are weighting coefficients.

b) Prioritized Client Update Scheduling: In the original GSFS, global aggregation occurs when a fixed proportion of client updates are received. We enhance this by prioritizing updates based on clients' historical contributions measured by model improvements. Formally, we define priority score P_k for client k as:

$$P_k = \alpha \cdot \overline{\Delta \mathcal{M}_k} + \beta \cdot (1/\tau_k), \tag{21}$$

where $\overline{\Delta \mathcal{M}_k}$ is the moving average of accuracy improvements, τ_k is the average time between updates from client k, and α, β are weighting parameters. Updates from clients with higher P_k values are aggregated first. While the original GSFS reduces communication overhead by triggering client uploads only when performance variation or gradient norms exceed predefined thresholds, it still relies on fixed thresholding and does not explicitly prioritize clients based on their historical contribution. In contrast, the proposed Enhanced GSFS introduces two key improvements: (i) adaptive threshold adjustment, where synchronization thresholds are dynamically updated based on recent accuracy improvement and communication frequency (Eq. (16)), and (ii) priority-aware client scheduling, where clients are ranked using a priority score reflecting their historical impact and update recency (Eq. (17)). These enhancements allow Enhanced GSFS to further reduce redundant updates and improve synchronization efficiency under heterogeneous IoMT environments.

D. Experimental Setup

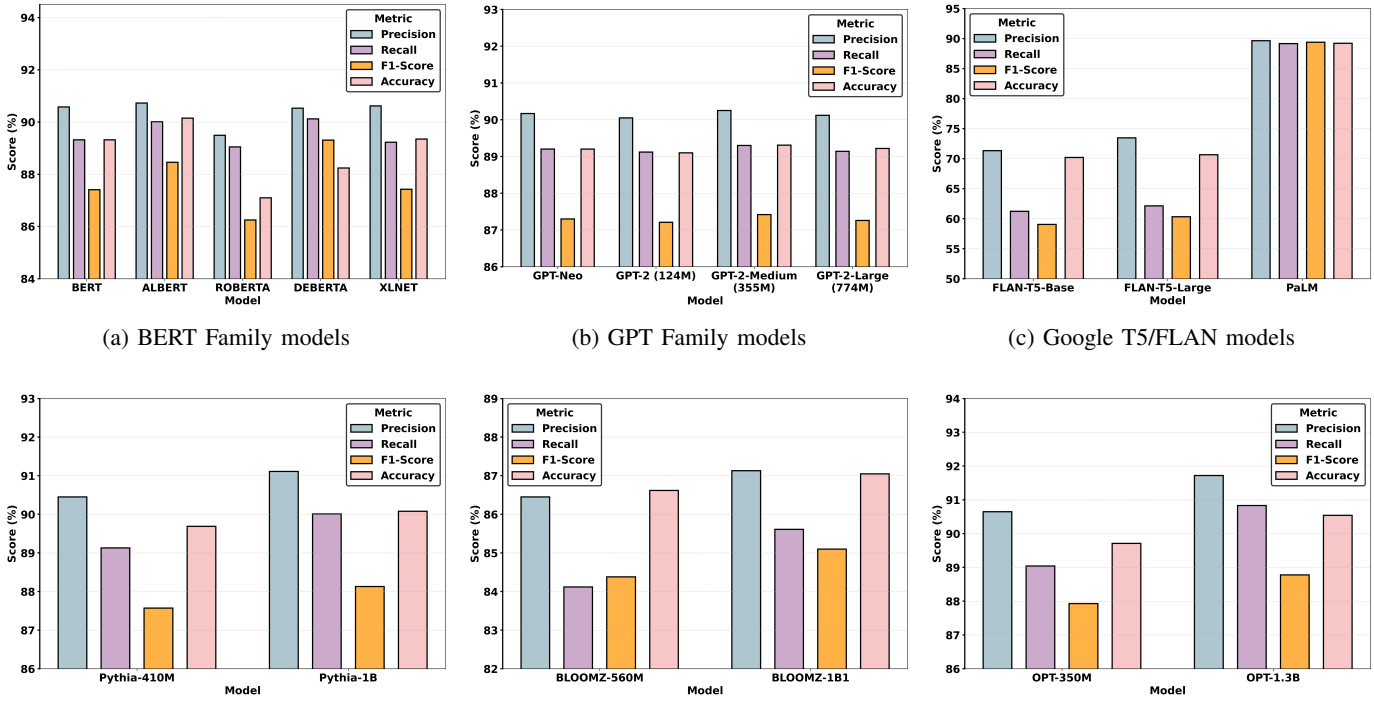
To evaluate the proposed framework, we conducted experiments on a two-tier federated platform that emulates realistic, compute-capable IoT deployment scenarios. The cloud tier (central infrastructure) was provisioned with high-performance computing resources, including an AMD EPYC server running Ubuntu 22.04 and equipped with an NVIDIA A100 GPU (40 GB of VRAM), which handles global model aggregation and coordination. The edge tier comprised five independent devices, each configured with 16 GB of system memory and an NVIDIA GeForce RTX 3080 Ti GPU (12 GB VRAM). The RTX 3080 Ti was selected to represent a cost-effective yet compute-capable edge GPU that is representative of practical hospital gateway and fog node deployments. More advanced GPUs (e.g., RTX 4090 or A-series accelerators) can also be deployed on-premises, which is expected to further improve inference speed, increase throughput, and enable local execution and adaptation of larger LLM variants. This would benefit end users by enabling faster real-time decision-making, reducing reliance on cloud connectivity, and improving responsiveness for latency-critical IoMT applications. Similarly, the framework can scale down to lower-power edge accelerators (e.g., NVIDIA Jetson-class devices) by deploying smaller LLM variants and applying lightweight adaptation (e.g., LoRA/prompt tuning), thereby trading off model capacity for reduced computational cost, lower energy consumption, and lower deployment cost. While such hardware exceeds the capabilities of traditional microcontroller-class IoT devices, it reflects the growing class of resource-rich edge environments, such as smart traffic hubs, industrial gateways, healthcare monitoring units, and fog computing platforms, that increasingly integrate mid-range GPUs to support advanced local intelligence. These edge nodes enable efficient on-site execution of distilled or size-optimized LLMs, allowing for semantic reasoning, anomaly detection, and contextual decision-making with reduced reliance on cloud services. The federated learning process was orchestrated using the proposed Enhanced GSFS, which adaptively regulates client participation based on real-time performance metrics. Latency and energy efficiency are reported for both tiers of the federated architecture. Specifically, the central measurements are obtained from the cloud aggregation server equipped with an NVIDIA A100 GPU, whereas the client measurements are obtained from edge devices equipped with NVIDIA RTX 3080 Ti GPUs. All models and federated strategies were evaluated under the same runtime and numerical-precision configuration to ensure a fair comparison. The experimental setup enabled a comprehensive assessment of the framework across key indicators, including accuracy, precision, recall, F1-score, inference latency, and energy efficiency. Both the IoT_23 and ToN_IoT datasets were used to benchmark system performance under heterogeneous, realistic operating conditions. The main hyperparameters for federated learning used in our experiments are summarized in Table IV.

IV. Results and Discussion

The proposed framework demonstrated significant improvements in IoT management tasks. Edge processing reduced latency by enabling real-time decision-making, while federated learning ensured data privacy and scalability. The adaptive feedback loop enhanced the system's ability to adjust to dynamic IoT environments by leveraging continuous performance monitoring and feedback. The monitoring dashboard provided administrators with actionable insights, enabling them to make quick adjustments and optimize system operations. Additionally, the distributed architecture enhanced resilience to failures and demonstrated scalability as the number of devices increased. Energy efficiency was significantly improved through localized processing on edge devices, thereby reducing reliance on cloud resources.

A. Model Performance Evaluation

We begin by evaluating the predictive performance of 17 transformer-based LLMs across two real-world IoT datasets: IoT-23 and ToN_IoT. The performance benchmarking of the 17 LLMs is conducted under a consistent evaluation setting to enable fair comparison across model families. The impact of federated learning strategies, including Enhanced GSFS, is evaluated separately in Section IV-B, where the selected base LLM model is trained under FedAvg, FedOpt, GSFS, and Enhanced GSFS for direct comparison. Although LLMs are primarily designed for generative tasks, in this work, they are adapted for sequence-level discriminative classification by converting structured IoT/IoMT telemetry records into text-like sequences using fixed templates. For encoder-based models (e.g., BERT), classification is performed using a softmax head applied to the pooled representation, whereas for



(a) BERT Family models (b) GPT Family models (c) Google T5/FLAN models

(d) EleutherAI models (e) BigScience BLOOM models (f) Meta OPT models

Fig. 2: Classification metrics for different LLM families.

decoder-based models (e.g., GPT), classification is achieved through prompt-based label generation. Model predictions are compared against ground-truth labels, and performance is evaluated using accuracy, precision, recall, and F1-score.

Fig. 2a illustrates the classification performance of five BERT-based models, BERT, ALBERT, ROBERTA, DEBERTA, and XLNET, across four key metrics: Precision, Recall, F1-score, and Accuracy. All models exhibit consistently high performance, with metric scores consistently exceeding 0.87, underscoring their robustness on the classification task. Among the evaluated models, ALBERT slightly outperforms the others in overall Precision (0.9073), Recall (0.9001), and Accuracy (0.9015), indicating its efficiency despite its parameter-reduction strategy. DEBERTA also delivers strong results, achieving the highest F1 Score (0.8931), indicating a good balance between precision and recall. BERT and XLNET, although earlier architectures, maintain stable performance, affirming their reliability for general-purpose NLP tasks. ROBERTA shows slightly lower F1-score (0.8625) and Accuracy (0.8710) than the others, which may be attributed to its sensitivity to task-specific data distributions. These results highlight that while all BERT-family models are capable performers, newer or optimized variants, such as ALBERT and DEBERTA, offer marginal yet meaningful improvements. This supports the motivation for refining and fine-tuning models, even within closely related Transformer architectures. Fig. 2b presents the classification performance of the GPT Family models, GPT-Neo, GPT-2 (124M), GPT-2-Medium (355M), and GPT-2-Large (774M), evaluated across four key metrics: Precision, Recall, F1-score, and Accuracy. All models achieve consistent scores in the range of 0.87-0.90, indicating strong generalization even without task-specific fine-tuning. GPT-2-Medium (355M) yields the highest F1-score (0.8742) among the group, reflecting its optimal balance between precision and recall. Interestingly, GPT-Neo, although architecturally distinct and more recent, exhibits recall (0.8920) and F1-score (0.8730) that are comparable to, though slightly lower than, those of its GPT-2 counterparts. A key observation is that scaling up the model size from 124M to 774M parameters in GPT-2 does not yield a linear improvement in classification performance. The largest model, GPT-2-Large (774M), achieves slightly lower F1-score (0.8726) and Precision (0.9012) than GPT-2-Medium. This suggests diminishing returns from larger model sizes for this task and dataset, potentially due to overparameterization or sensitivity to input length and context window. Overall, the GPT Family models demonstrate solid performance in text classification, with mid-sized models, such as GPT-2-Medium, offering an effective trade-off between computational cost and accuracy. This finding is valuable for deployment in IoT environments, where model efficiency is critical. The performance comparison shown in Fig. 2c highlights notable differences among the Google T5/FLAN models. Both Google/flan-t5-base and Google/flan-t5-large demonstrate relatively low performance across all classification metrics, precision, recall, F1-score, and accuracy, generally ranging between 0.60 and 0.73. This suggests that these models struggle to produce consistent predictions, particularly in recall, indicating a higher rate of false negatives. In contrast, the PALM model achieves significantly superior results across the board, with all metrics consistently above 0.89. This indicates that PALM offers substantially better generalization, precision, and overall reliability than the

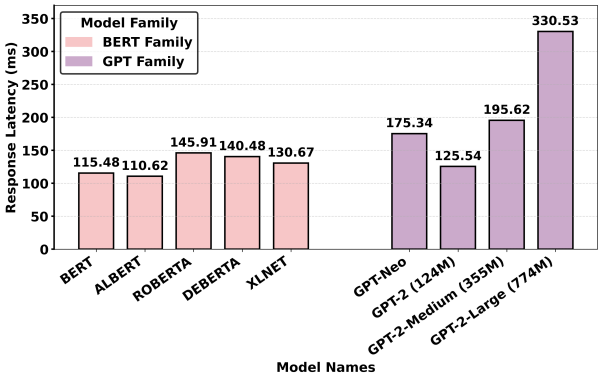


Fig. 3: Response Latency for BERT and GPT Family Models

earlier FLAN variants. The significant performance improvement highlights the impact of model size and architectural enhancements introduced in PALM, making it more suitable for high-stakes classification tasks where precision and recall are crucial.

Fig. 2d illustrates the classification metrics for the EleutherAI models: EleutherAI/pythia-410M and EleutherAI/pythia-1b. Both models exhibit strong and consistent performance across all evaluated metrics, precision, recall, F1-score, and accuracy, with values that are close to or exceed 0.90. Notably, EleutherAI/pythia-1b slightly outperforms the 410M variant across all metrics, reflecting the advantages of its larger model capacity. The differences, while modest, indicate improved generalization and robustness in the pythia-1b model. This suggests that increasing the model size within the EleutherAI architecture enhances classification performance without compromising the balance between precision and recall. Overall, both models demonstrate high effectiveness and are well-suited for classification tasks in performance-critical applications.

Fig. 2e presents the classification performance of the BigScience BLOOM models: bloomz-560m and bloomz-1b1. Both models deliver strong and consistent results across the four evaluated metrics, precision, recall, F1-score, and accuracy, with scores clustering around or slightly above 0.85. Between the two, bloomz-1b1 consistently outperforms bloomz-560m, albeit by a small margin. This performance gain highlights the benefits of increased model capacity, likely due to improved contextual understanding and representation learning in the larger model. The precision and accuracy metrics show particular strength, indicating that these models are well calibrated to minimize false positives and deliver reliable overall predictions. The near balance between precision and recall in both models suggests robustness across different class distributions. These characteristics make the BLOOM models viable candidates for multilingual or large-scale classification tasks in diverse settings. Fig. 2f presents the classification performance metrics for the Meta OPT models, specifically OPT-350M and OPT-1.3b. Overall, both models demonstrate high and balanced performance across all four metrics, consistently exceeding 0.87. OPT-1.3b outperforms OPT-350M across all metrics, reflecting the expected benefit of increased model capacity. Notably, OPT-1.3b achieves slightly higher values in Precision (0.9172), Recall (0.9083), and F1-score (0.8878), compared to 0.9065, 0.8904, and 0.8793, respectively, for OPT-350M. Accuracy for OPT-1.3b also reaches 0.9054, compared to 0.8971 for the smaller counterpart. The proximity of these scores indicates that both models maintain strong generalization and robustness. The narrow gap between Precision and Recall across models implies that neither overfitting nor underfitting is dominant. These results suggest that the Meta OPT family, particularly the 1.3B-parameter variant, is a highly reliable choice for classification tasks in resource-rich deployments. Figs. 3 and 4 provide a comparison of the response latency across various LLM families. The latency values reflect the time each model takes to generate a prediction, which is critical for real-time or latency-sensitive applications in edge or IoT environments. In Fig. 3, the BERT family exhibits moderate latency, with ROBERTA and DEBERTA showing the highest latencies at 145.91 ms and 140.48 ms, respectively. ALBERT demonstrates the lowest latency (110.62 ms), consistent with its lightweight, efficient architecture. In addition, the GPT family shows a noticeable increase in latency as model size scales. GPT-2-Large (774M) records the highest latency (330.53 ms), while the smallest model, GPT-2 (124M), achieves a lower latency of 125.54 ms. GPT-Neo, despite being relatively lightweight, has a higher latency (175.34 ms), likely due to architectural differences. As shown in Fig. 4, the Google T5/FLAN models exhibit a wide range of latencies. The flan-t5-base variant performs significantly better (95.48 ms) than flan-t5-large (165.96 ms) and PALM (165.83 ms). The increased latency of larger models is attributable to their deeper, wider transformer layers. Similarly, for EleutherAI models, pythia-1b incurs a considerably higher latency (280.62 ms) than pythia-410M (145.42 ms). This nearly twofold difference aligns with the increase in parameter scale and confirms the impact of model complexity on response time. Moreover, BigScience BLOOM models follow a similar trend. The 1.1B model incurs a latency of 310.84 ms, nearly double that of the smaller 560M variant (170.48 ms). This disparity underscores the computational overhead introduced by larger model variants. Finally, for Meta OPT models, OPT-1.3b reaches 215.36 ms, significantly higher than OPT-350M (88.14 ms). Despite both models sharing the same architectural principles, the difference in parameter count directly affects inference speed. Overall, these figures emphasize a consistent trade-off between model size and response latency across all families. Smaller models deliver faster predictions and are better suited to edge deployment, whereas larger models, although more accurate, require substantially higher latency budgets. This latency-performance balance is crucial in selecting an appropriate model for constrained environments.

Fig. 5 presents the energy efficiency of the evaluated transformer-based models, measured in requests per minute (req/min). This metric is crucial in assessing the models' suitability for deployment in energy-constrained environments, such as edge devices or large-scale production systems. As shown in Fig. 5, the BERT family demonstrates consistently high energy efficiency. ALBERT and BERT lead with 542.50 and 519.40 req/min, respectively, thanks to their lighter architectures and optimization strategies. DEBERTA and XLNET

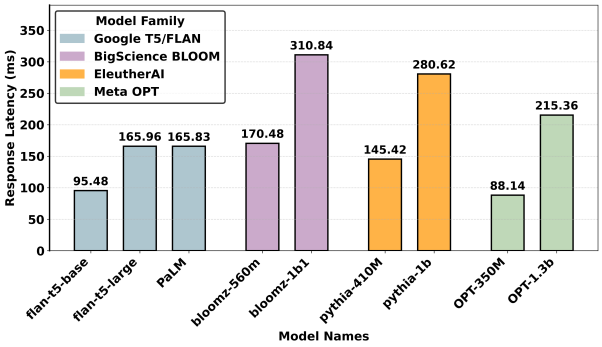


Fig. 4: Response Latency for Google T5/FLAN Models, EleutherAI Models, BigScience BLOOM Models, and Meta OPT Models

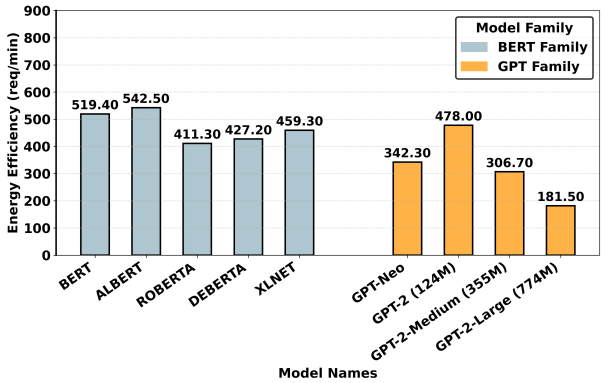


Fig. 5: Energy Efficiency for BERT and GPT Family Models

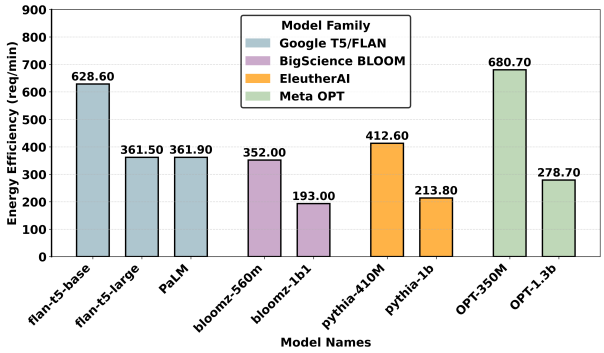


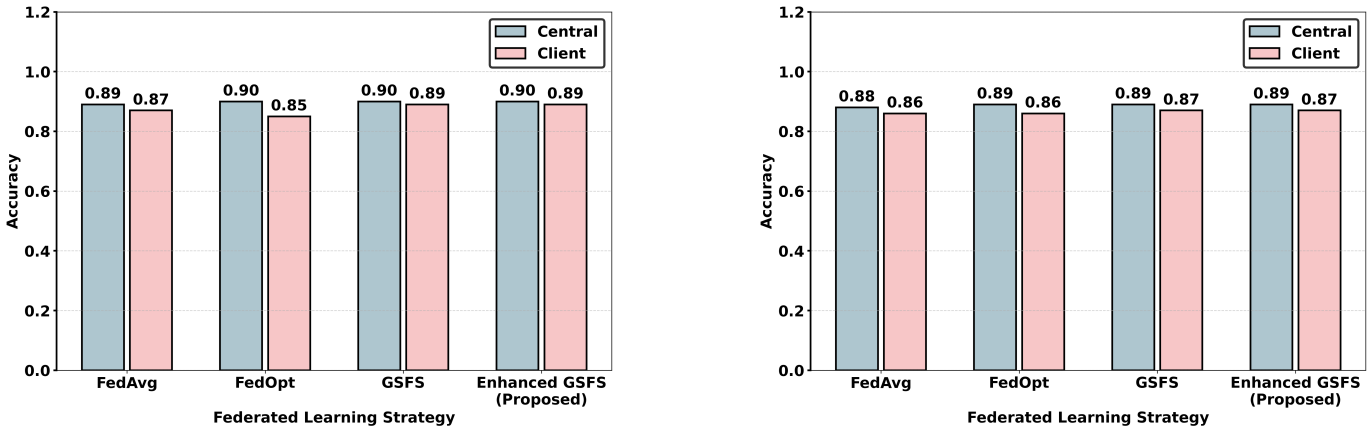
Fig. 6: Energy Efficiency for for Google T5/FLAN Models, EleutherAI Models, BigScience BLOOM Models, and Meta OPT Models

also maintain competitive throughput above 400 req/min, while ROBERTA lags slightly at 411.30 req/min. The GPT family demonstrates a sharp decline in efficiency as model size increases. GPT-2 (124M) achieves the highest efficiency within its family at 478.00 req/min, while GPT-2-Large (774M) drops drastically to 181.50 req/min. This decline underscores the computational overhead of large-scale autoregressive models during sequential token generation. Fig. 6 illustrates the energy efficiency of Google T5/FLAN Models, EleutherAI Models, BigScience BLOOM Models, and Meta OPT Models. The Google T5/FLAN family achieves the highest overall efficiency, with FLAN-T5-base reaching 628.60 req/min, the peak among all models evaluated. However, efficiency decreases for FLAN-T5-large and PALM, each stabilizing around 361 req/min, indicating a steep trade-off associated with increased model capacity. EleutherAI models follow a predictable scale-based trend. The smaller Pythia-410M model achieves 412.60 req/min, significantly outperforming the larger Pythia-1B, which records 213.80 req/min. For BigScience BLOOM models, the 560M variant performs better, at 352.00 req/min, whereas the 1b1 model shows a notable drop to 193.00 req/min, suggesting energy inefficiencies for larger multilingual generation tasks. Finally, the Meta OPT models demonstrate remarkable efficiency, with OPT-350M achieving the highest throughput overall at 680.70 requests per minute. This positions it as the most energy-efficient model in the study. The larger OPT-1.3b model exhibits a reduced rate of 278.70 req/min, consistent with the observed inverse correlation between model size and energy efficiency across families. These results affirm that while larger models often offer superior accuracy and generalization, they incur significant energy costs, which may limit their viability for real-time or large-scale deployment. Mid-sized models, particularly OPT-350M and FLAN-T5-base, offer an optimal balance between performance and computational cost.

B. Federated Learning Strategy Comparison

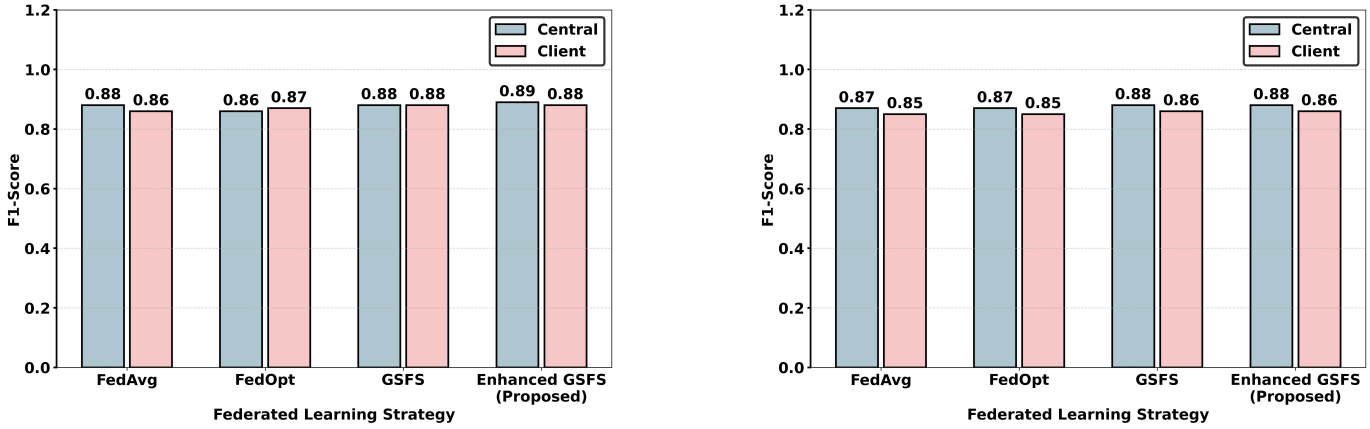
To evaluate the benefits of our proposed Enhanced GSFS strategy, we compare it against multiple baseline methods, including FedAvg, FedOpt, and the original GSFS. We analyze their performance with respect to synchronization frequency, model convergence, latency, energy consumption, and communication overhead, highlighting the impact of adaptive feedback- and reinforcement-learning-driven control in federated coordination. In addition to latency and energy profiling, we quantify communication overhead by measuring the total number of client uploads and the cumulative size of transmitted updates (MB) per federated round. This enables direct comparison of synchronization cost across FedAvg, FedOpt, GSFS, and Enhanced GSFS. Although our evaluation is conducted on IoT-23 and ToN_IoT, the proposed FL–LLM framework is not dataset-specific and can generalize to other IoT/IoMT security datasets, as it relies on serialized telemetry representations and federated optimization. Consistent trends across two heterogeneous datasets indicate the robustness of Enhanced GSFS across varying data distributions. This section also presents the quantitative results and evaluation curves used to select the base LLM model. To be more specific, two of the performance metrics were defined as follows: **Response Latency**: the average inference time required for the model to process the test set; and **Energy Efficiency**: the number of requests the model can handle within one minute.

Fig. 7a presents the classification accuracy of both the central and client models across four federated learning strategies: FedAvg, FedOpt, GSFS, and the proposed Enhanced GSFS. This evaluation on the IoT-23 dataset provides insight into the trade-offs between global model performance and local client effectiveness. The FedAvg baseline achieves balanced performance, with the central model achieving 0.89 accuracy



(a) Accuracy on IoT-23 (b) Accuracy on ToN_IoT

Fig. 7: Comparison of accuracy across IoT-23 and ToN_IoT datasets using different federated learning strategies.



(a) F1-score on IoT-23 (b) F1-score on ToN_IoT

Fig. 8: Comparison of F1-score across IoT-23 and ToN_IoT datasets using different federated learning strategies.

and the client model slightly lower at 0.87. While this indicates acceptable convergence, it also highlights minor client-level degradation due to heterogeneous data distributions. FedOpt improves the central model accuracy to 0.90 but suffers a significant drop in client accuracy to 0.85. This disparity suggests that while global optimization improves server-side performance, it may amplify model divergence across clients, particularly in non-IID environments such as IoT security datasets. The GSFS strategy delivers a more balanced accuracy profile, achieving 0.90 and 0.89 for the central and client models, respectively. This narrow gap suggests enhanced synchronization between global and local updates, likely due to GSFS's gradient-sharing mechanism, which mitigates client drift. The proposed Enhanced GSFS further refines this behaviour, maintaining a central accuracy of 0.90 and a client accuracy of 0.89. These results confirm that the enhancements introduced in our approach not only sustain high global performance but also preserve robust client-side generalization. Overall, the Enhanced GSFS strategy demonstrates superior consistency and balance across the federated architecture, making it more suitable for deployment in distributed IoT systems where equitable client performance is critical.

Fig. 7b illustrates the comparative accuracy of central and client models for four federated learning strategies on the ToN_IoT dataset. This dataset exhibits a distinct distribution and complexity profile compared with IoT-23, enabling assessment of generalizability across various IoT environments. The FedAvg strategy shows a modest central accuracy of 0.88 and a slightly lower client accuracy of 0.86. This gap indicates limited adaptation to client-specific data distributions and potential challenges with model personalization. FedOpt achieves a marginally higher central accuracy of 0.89; however, similar to the IoT-23 results, its client accuracy drops to 0.86. This again suggests that while optimization accelerates global convergence, it may neglect local adaptability, especially in heterogeneous settings. Both GSFS and Enhanced GSFS deliver a more balanced outcome. GSFS achieves an accuracy of 0.89 for both the central and client models, underscoring its stability and effectiveness in harmonizing gradient updates across the federation. Enhanced GSFS maintains this equilibrium, with a central accuracy of 0.89 and a slightly lower client accuracy of 0.87. The results demonstrate that Enhanced GSFS consistently narrows the performance gap between central and client models, improving generalization

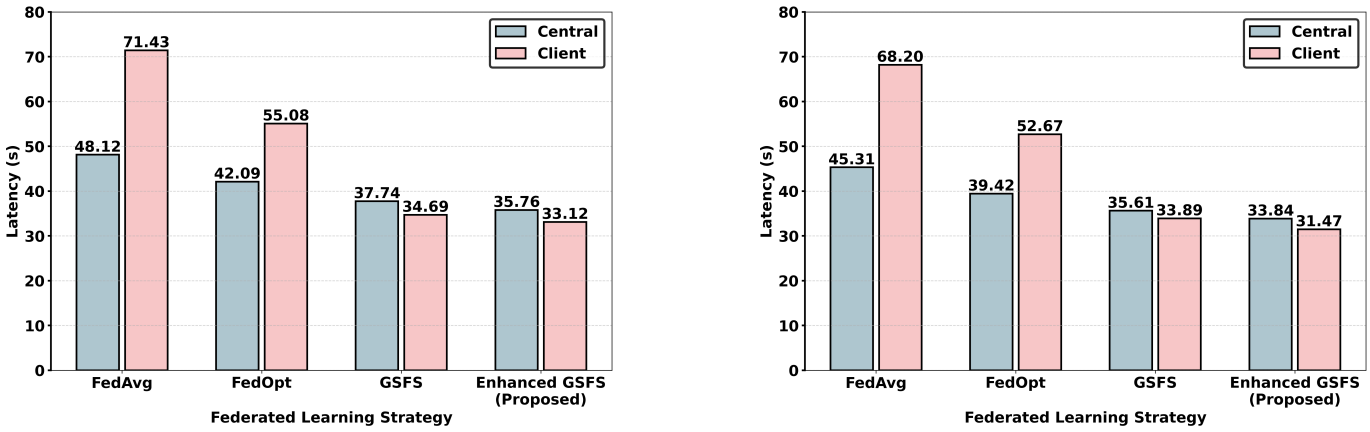
while maintaining robust performance for individual clients. This is particularly crucial in practical federated IoT deployments where model fairness and reliability across nodes are essential. Fig. 8a presents the F1-score performance of central and client models trained using the four federated learning strategies on the IoT-23 dataset. F1-score, as the harmonic mean of precision and recall, provides a robust measure of classification effectiveness for imbalanced datasets, which are common in cybersecurity-related IoT contexts. Both FedAvg and GSFS achieve balanced F1 Scores of 0.88 between the central and client models, indicating consistent predictive performance across the federation. However, FedOpt reveals a slight central-client performance gap (0.86 vs. 0.87), reflecting instability in local optimization dynamics. Enhanced GSFS outperforms the baselines, achieving an F1-score of 0.89 on the central model and 0.88 on the clients. This marginal yet consistent improvement underscores Enhanced GSFS's capacity to coordinate learning across heterogeneous clients, ensuring generalizability without sacrificing individual performance. These findings reinforce the utility of Enhanced GSFS in real-world federated scenarios, where both accuracy and fairness are critical across diverse IoT devices. Fig. 8b illustrates the F1-score performance of both central and client models for the four federated learning strategies, FedAvg, FedOpt, GSFS, and the proposed Enhanced GSFS, on the ToN_IoT dataset. FedAvg and FedOpt both achieve identical F1 Scores of 0.87 and 0.85 for central and client models, respectively, indicating a consistent performance gap of 0.02. GSFS yields slightly better results, with F1 Scores of 0.88 and 0.86, indicating improved coordination between the central and distributed models. Notably, the Enhanced GSFS strategy maintains the highest central-model F1 score of 0.88 and achieves 0.86 at the client level. This marginal gain highlights Enhanced GSFS's ability to maintain high predictive performance while minimizing discrepancies between the central and local models. The results on the ToN_IoT dataset confirm the advantage of Enhanced GSFS in federated learning environments by offering both stability and accuracy across model deployments.

C. Latency Profiling Evaluation

In latency-sensitive IoMT environments, predictive accuracy alone is insufficient; deployment viability also depends on communication delay and system responsiveness. This subsection presents a detailed latency-profiling analysis of the four federated learning strategies: FedAvg, FedOpt, GSFS, and the proposed Enhanced GSFS, evaluated on both the IoT-23 and ToN_IoT datasets. By comparing runtime latency on both central and client nodes, we assess each method's suitability for real-time IoMT applications and highlight the improvements introduced by Enhanced GSFS. In this work, latency is defined as the average inference time per sample measured over the full test set. Specifically, we measure the total wall-clock time required to process the entire test set and divide it by the number of test samples. All models and federated strategies are evaluated using the same execution environment to ensure fair comparison. Fig. 9a presents latency measurements (in seconds) for both the central and client models across the four federated learning strategies using the IoT-23 dataset. FedAvg incurs the highest latency, with central and client latencies recorded at 48.12s and 71.43s, respectively. This substantial gap highlights FedAvg's inefficiency in synchronous communication and aggregation, particularly from the client side. FedOpt improves on this, reducing latency to 42.09 seconds (central) and 55.08 seconds (client), though the disparity between the two remains significant. GSFS achieves lower latency, with central and client times of 37.74s and 34.69s, respectively, indicating more balanced computational and communication overhead. The proposed Enhanced GSFS further improves efficiency, yielding the lowest latencies of 35.76 seconds (central) and 33.12 seconds (client). This demonstrates the benefit of the enhanced coordination and scheduling mechanisms embedded in our approach. In summary, Enhanced GSFS outperforms all baseline methods in latency minimization, particularly by narrowing the central-client latency gap, which is critical for real-time IoT deployments. Fig. 9b illustrates the latency performance (in seconds) of both central and client models across four federated learning strategies evaluated on the ToN_IoT dataset. A clear trend of improved communication efficiency is evident as we transition from traditional aggregation methods (FedAvg and FedOpt) to the proposed Enhanced GSFS. Specifically, FedAvg exhibits the highest latency, with the client-side latency peaking at 68.20 seconds and central latency at 45.31 seconds. FedOpt slightly improves this latency profile, reducing the client and central times to 52.67 and 39.42 seconds, respectively. The GSFS strategy achieves notable latency reductions to 33.89 seconds (client) and 35.61 seconds (central), highlighting its efficiency in reducing communication overhead. The proposed Enhanced GSFS achieves the lowest latency across both perspectives, with client latency dropping to 31.47 seconds and central latency to 33.84 seconds, highlighting the strategy's effectiveness in minimizing response delays. This improvement is attributed to its optimized aggregation mechanism and intelligent scheduling, which collectively reduce synchronization overhead. These results confirm that Enhanced GSFS not only maintains competitive accuracy and F1-score performance but also significantly enhances communication efficiency, which is crucial for real-time applications in IoT environments.

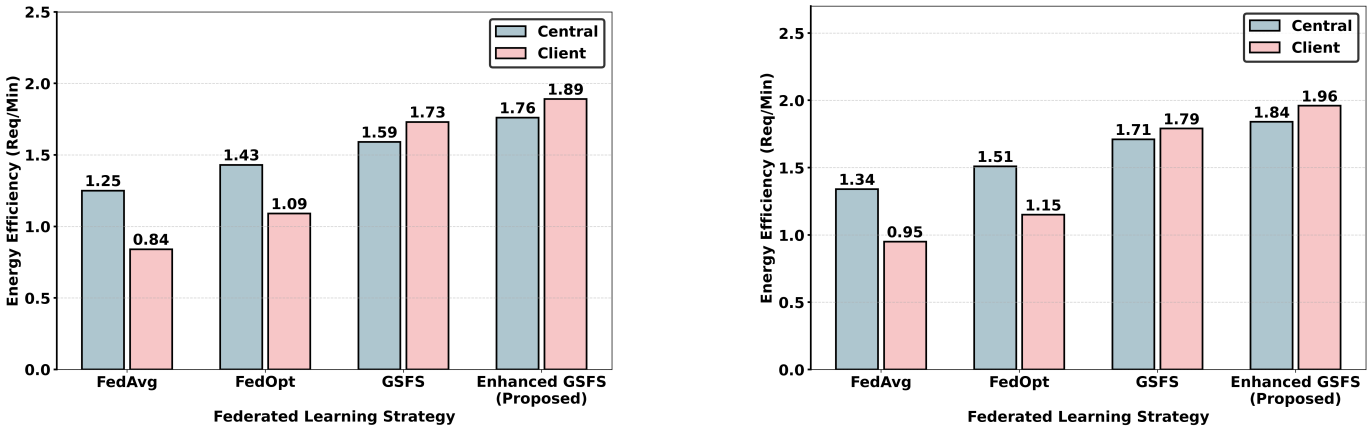
D. Energy Profiling Evaluation

Energy efficiency is a critical metric for sustainable IoT system design, particularly in federated settings where distributed training incurs nontrivial energy overhead. This subsection evaluates the energy performance of FedAvg, FedOpt, GSFS, and Enhanced GSFS, using the IoT-23 and ToN_IoT datasets. By measuring throughput in requests per minute (Req/min), we assess the trade-offs between computational workload and energy utilization, and demonstrate the impact of Enhanced GSFS on energy-aware federated learning. Fig. 10a compares the energy efficiency of the central and client models across four federated learning strategies on the IoT-23 dataset. The proposed Enhanced GSFS approach significantly outperforms the baseline methods in terms of requests per minute. Specifically, Enhanced GSFS achieves the highest energy efficiency with 1.76 and 1.89 requests per minute for the central and



(a) Latency on IoT-23 (b) Latency on ToN_IoT

Fig. 9: Comparison of latency across IoT-23 and ToN_IoT datasets using different federated learning strategies.



(a) Energy efficiency on IoT-23 (b) Energy efficiency on ToN_IoT

Fig. 10: Comparison of energy efficiency across IoT-23 and ToN_IoT datasets using different federated learning strategies.

client models, respectively. This indicates its capability to maintain a high throughput while minimizing energy consumption. In contrast, traditional methods such as FedAvg and FedOpt lag, with FedAvg exhibiting the lowest efficiency, particularly on the client side (0.84 requests per minute). FedOpt demonstrates a moderate improvement, achieving rates of 1.43 and 1.09 requests per minute for the central and client models, respectively. GSFS outperforms these baseline methods, achieving 1.59 (central) and 1.73 (client) requests per minute; however, it still falls short of Enhanced GSFS. The observed improvements can be attributed to the optimized gradient selection and synchronization scheme introduced in Enhanced GSFS, which reduces redundant computation and communication. This leads to a more sustainable and scalable FL deployment, especially relevant for energy-constrained IoT environments. The consistent energy advantage across both central and client models confirms the robustness of the proposed method in diverse deployment scenarios. Fig. 10b presents the comparative analysis of energy efficiency, measured in requests per minute (Req/min), for central and client models across four federated learning strategies, on the ToN_IoT dataset. This figure illustrates the key trade-offs between computational performance and energy utilization in distributed training environments. The results reveal that Enhanced GSFS outperforms all other strategies, achieving the highest energy efficiency: 1.84 Req/min for the central model and 1.96 Req/min for the client model. This improvement is attributable to the strategy's ability to minimize redundant communication and optimize local updates, thereby reducing idle time and improving throughput. Among the baseline strategies, GSFS consistently shows higher energy efficiency than FedAvg and FedOpt, with central and client scores of 1.79 and 1.71 Req/min, respectively. This supports the effectiveness of global sparsity-based synchronization in conserving computational resources while maintaining performance. Conversely, FedAvg and FedOpt exhibit lower energy efficiency, particularly in the client models, with FedAvg-client achieving the lowest at 0.95 Req/min. The central models under these strategies also show relatively reduced values of 1.34 and 1.51 Req/min for FedAvg and FedOpt, respectively. This disparity indicates higher communication overhead and less optimized training cycles.

The ablation results in Table V show that the proposed enhancements mainly improve system efficiency while pre-

TABLE V: Performance Analysis of Enhanced GSFS Components on IoT-23 and ToN-IoT

Dataset	Variant	Accuracy		F1-score		Latency (s) ↓		Energy (Req/min) ↑		Communication Cost ↓	
		Central	Client	Central	Client	Central	Client	Central	Client	Uploads/Round	MB/Round
IoT-23	GSFS (base)	0.90	0.89	0.88	0.88	37.74	34.69	1.59	1.73	3.10	18.40
	GSFS + AT only	0.90	0.89	0.88	0.88	36.80	33.95	1.66	1.80	2.85	16.90
	GSFS + PS only	0.90	0.89	0.88	0.88	37.10	34.10	1.63	1.77	2.92	17.40
	Enhanced GSFS (AT+PS)	0.90	0.89	0.89	0.88	35.76	33.12	1.76	1.89	2.60	15.20
ToN-IoT	GSFS (base)	0.89	0.87	0.88	0.86	35.61	33.89	1.71	1.79	3.00	17.60
	GSFS + AT only	0.89	0.87	0.88	0.86	34.95	33.10	1.76	1.85	2.75	16.20
	GSFS + PS only	0.89	0.87	0.88	0.86	35.20	33.35	1.74	1.83	2.82	16.70
	Enhanced GSFS (AT+PS)	0.89	0.87	0.88	0.86	33.84	31.47	1.84	1.96	2.50	14.80

serving predictive performance. For both IoT-23 and ToN-IoT datasets, the accuracy and F1-score remain almost unchanged across all variants, indicating that Adaptive Thresholding (AT) and Priority Scheduling (PS) do not degrade the model's classification performance. At the same time, both AT and PS reduce latency and communication cost compared with the original GSFS. On IoT-23, GSFS+AT achieves lower latency and communication overhead than the base GSFS, while GSFS+PS also provides consistent improvements. A similar trend is observed on ToN-IoT. This shows that each component contributes positively to synchronization efficiency. The full Enhanced GSFS, which combines AT and PS, gives the best overall results on both datasets. It achieves the lowest central and client latency, the highest energy efficiency, and the lowest communication cost in terms of both uploads per round and transmitted data size. These results suggest that the two components complement each other well. This confirms that the gains of Enhanced GSFS come mainly from improved coordination and communication efficiency, while maintaining stable predictive performance. One limitation of the current study is that the evaluation does not explicitly account for adversarial attacks in federated learning, such as poisoning or backdoor manipulation by malicious clients. The present experiments focus on standard non-IID federated conditions and system-level trade-offs in accuracy, latency, energy efficiency, and communication cost. Evaluating the resilience of Enhanced GSFS under adversarial client behaviour is an important direction for future work.

V. Conclusion

This paper presents a scalable, privacy-preserving architecture for deploying LLMs in federated IoMT environments that integrates edge-level fine-tuning at hospital gateways and clinical fog nodes, prompt adaptation, and cloud-based coordination. The proposed framework addresses key challenges in latency, energy efficiency, and HIPAA/PHIPA-aligned data privacy. Extensive experiments on IoT-23 and ToN_IoT demonstrate the effectiveness of Enhanced GSFS in improving inference performance across multiple transformer families under realistic edge–cloud settings. Despite these promising results, this work has three main limitations. First, due to privacy restrictions and the limited availability of public IoMT attack traces, our evaluation relies on the IoT-23 and ToN_IoT datasets as proxies. Second, robustness to adversarial threats in federated learning, such as poisoning and backdoor attacks, was not explicitly evaluated. Third, an ablation study isolating the individual contributions of adaptive thresholding and priority-aware client scheduling was not included.

References

[1] A. Brown, R. Singh, and Y. Zhang, "Heterogeneous iot systems: Addressing diversity in devices and networks," *IoT Journal*, vol. 8, no. 2, pp. 112–125, 2024.

[2] Y. Otoum, P. Singh, and A. Nayak, "Advancing iomt defenses: Deep collaborative learning for robust healthcare security," in *GLOBECOM 2024-2024 IEEE Global Communications Conference*. IEEE, 2024, pp. 2966–2971.

[3] H. Lee, M. Garcia, and S. Patel, "Integrating large language models at the edge for iot applications," *ACM Transactions on Embedded Computing Systems*, vol. 23, no. 5, pp. 150–167, 2024.

[4] S. Patel and A. Kumar, "Energy-efficient iot: Balancing computational workloads in edge-cloud architectures," *IEEE Internet of Things Journal*, vol. 11, no. 6, pp. 789–801, 2024.

[5] M. Johnson and L. Zhao, "Critical iot applications: Overcoming latency and failure risks," *IEEE Transactions on Industrial Informatics*, vol. 21, no. 1, pp. 78–89, 2025.

[6] Y. Otoum, A. Asad, and A. Nayak, "Llms meet federated learning for scalable and secure iot management," *arXiv preprint arXiv:2504.16032*, 2025.

[7] L. Jin, Y. Zhang, Y. Li, S. Wang, H. H. Yang, J. Wu, and M. Zhang, "Moe ˆ2: Optimizing collaborative inference for edge large language models," *arXiv preprint arXiv:2501.09410*, 2025.

[8] Z. Fang, Y. Zhang, L. Wang *et al.*, "Fate-llm: An industrial-grade federated learning framework for large language models," *arXiv preprint arXiv:2310.10049*, 2023.

[9] M. Xu, P. Wang *et al.*, "Fwdllm: Efficient federated fine-tuning of large language models," in *USENIX Annual Technical Conference (ATC)*, 2024.

[10] T. Zhao *et al.*, "Koala: Federated knowledge distillation for resource-constrained llm clients," *arXiv preprint arXiv:2407.05268*, 2024.

[11] A. Alamleh, O. S. Albahri, A. A. Zaidan, A. S. Albahri, A. H. Alamoodi, B. B. Zaidan, S. Qahtan, H. A. Alsatar, M. S. Al-Samarraay, and A. N. Jasim, "Federated learning for iomt applications: A standardization and benchmarking framework of intrusion detection systems," *IEEE Journal of Biomedical and Health Informatics*, vol. 27, no. 2, pp. 878–887, 2023.

[12] B. Bhasker, P. M. Rao, P. Saraswathi, S. G. K. Patro, J. K. Bhutto, S. Islam, M. Kareemullah, and A. F. Emma, "Blockchain framework with iot device using federated learning for sustainable healthcare systems," *Scientific Reports*, vol. 15, p. 26736, 2025.

[13] R. U. Z. Wani and O. Can, "Fed-ehr: A privacy-preserving federated learning framework for decentralized healthcare analytics," *Electronics*, vol. 14, no. 16, p. 3261, 2025.

[14] F. R. Albogamy, "Federated learning for iomt-enhanced human activity recognition with hybrid lstm-gru networks," *Sensors*, vol. 25, no. 3, p. 907, 2025.

[15] A. M. Almogadwy and A. A. Alqarafi, "Fused federated learning framework for secure and decentralized patient monitoring in healthcare 5.0 using iomt," *Scientific Reports*, vol. 15, p. 24263, 2025.

[16] R. A. Alharbey and F. Jamil, "Federated learning framework for real-time activity and context monitoring using edge devices," *Sensors*, vol. 25, no. 4, p. 1266, 2025.

[17] Z. Wang, Z. Shen, Y. He, G. Sun, H. Wang, L. Lyu, and A. Li, "Flora: Federated fine-tuning large language models with heterogeneous low-rank adaptations," in *Proceedings of NeurIPS 2024*, 2024.

[18] R. Ye, W. Wang, J. Chai, D. Li, Z. Li, Y. Xu, Y. Du, Y. Wang, and S. Chen, "Openfedllm: Training large language models on decentralized private data via federated learning," *arXiv preprint*, vol. arXiv:2402.06954, 2024.

[19] H. Ding, Y. Yan, Y. Lu, J.-H. Xue, and H. Wang, "Fcllm-dt: Empowering federated continual learning with large language models for industrial iot," *IEEE Transactions on Circuits and Systems for Video Technology*, 2025.

[20] X. Zhang, Y. Liu, Y. Kang, L. Li, T. Chen, M. Hong, and Q. Yang, "Towards federated large language models: Motivations, methods, and open challenges," *IEEE Transactions on Neural Networks and Learning Systems*, 2024.

[21] X. Ma, F. Fang, and X. Wang, "Dynamic authentication and granularized authorization with a cross-domain zero-trust architecture for federated learning in large-scale iot networks," *arXiv preprint*, vol. 2501.03601, 2025.

[22] K. Li, C. Li, X. Yuan, S. Li, S. Zou, S. S. Ahmed, W. Ni, D. Niyato, A. Jamalipour, F. Dressler, and O. B. Akan, "Zero-trust foundation models: A new paradigm for secure and collaborative artificial intelligence for internet of things," *arXiv preprint*, vol. 2505.23792, 2025.

[23] A. Sharma, S. Rani, and W. Boulila, "Blockchain-based zero-trust networks with federated transfer learning for iot security in industry 5.0," *PLOS ONE*, vol. 20, no. 6, p. e0323241, 2025.

[24] T. An, Y. Zhou, H. Zou, and J. Yang, "Iot-llm: Enhancing real-world iot task reasoning with large language models," *arXiv preprint arXiv:2410.02429*, 2024.

[25] Y. Yigit, M. A. Ferrag, M. C. Ghanem, I. H. Sarker, L. A. Maglaras, C. Chrysoulas, N. Moradpoor, N. Tihanyi, and H. Janicke, "Generative ai and llms for critical infrastructure protection: Evaluation benchmarks, agentic ai, challenges, and opportunities," *Sensors*, vol. 25, no. 6, p. 1666, 2025.

[26] F. Adjewa, M. Esseghir, and L. Merghem-Boulahia, "Efficient federated intrusion detection in 5g ecosystem using optimized bert-based model," in *2024 20th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob)*. IEEE, 2024, pp. 62–67.

[27] C. Mawela, C. B. Issaid, and M. Bennis, "A web-based solution for federated learning with llm-based automation," *IEEE Internet of Things Journal*, 2025.

[28] R. Mahmud, R. Kotagiri, and R. Buyya, "Fog computing: A taxonomy, survey and future directions," *IEEE Internet of Things Journal*, vol. 6, no. 3, pp. 4048–4072, 2019.

[29] N. Rathore *et al.*, "Intelligent edge–fog interplay for healthcare informatics," *Computer Communications*, 2024, ahead-of-print.

[30] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[31] Y. Otoum and A. Nayak, "Differential privacy-driven framework for enhancing heart disease prediction," in *ICC 2025-IEEE International Conference on Communications*. IEEE, 2025, pp. 5456–5462.

[32] T. Dierks and E. Rescorla, "The transport layer security (tls) protocol version 1.2," RFC 5246, Internet Engineering Task Force (IETF), 2008. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5246

[33] M. Gao, B. Sun, Z. Fan, T. Zhang, and Z. Zheng, "Domain-aware reinforcement learning for prompt optimization," *Mathematics*, vol. 13, no. 16, p. 2552, 2025.

[34] M. Deng, J. Wang, C.-J. Hsieh, Y. Wang, H. Guo, T. Shu, M. Song, E. P. Xing, and Z. Hu, "Rlprompt: Optimizing discrete text prompts with reinforcement learning," *arXiv preprint arXiv:2205.12548*, 2022.

[35] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "Llmlingua: Compressing prompts for accelerated inference of large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, 2023, pp. 13 358–13 376.

[36] S. Garcia, A. Parmisano, and M. J. Erquiaga, "Iot-23: A labeled dataset with malicious and benign iot network traffic," *(No Title)*, 2020.

[37] M. Gharib, J. Hu, and M. Alazab, "Ton_iot: A realistic and diverse dataset for evaluating iot intrusion detection," *IEEE Internet of Things Journal*, vol. 8, no. 15, pp. 12 399–12 410, 2021.